

DIGITAL TWIN CATALOG

System Architecture Document

Structure, deployment, data flow, security, risks and architecture decisions.

Project	Digital Twin Catalog (NARAGA 3D Catalog), reusable 3DCityDB template
Document	System Architecture
Version	0.1 draft for review, 4 October 2026
Author	Fairuz Akmal Pradana
Repository	github.com/fakmalpradana/naraga-twin-db

Contents

Introduction

Purpose, scope, stakeholders

Requirements

Component view (C4 container)

Deployment view

Data flow

Architecture decisions (ADR)

Security and access

Risks and deferred

Introduction

Status: **draft for review** · 2026-10-03 · Related: [DDD](#), [ERD](#), [Schema](#), [API](#), [Deployment](#), [Runbook](#)

Purpose, scope, stakeholders

A long-lived, reusable **digital twin database + catalog** on 3DCityDB v5. It stores semantic CityGML 2.0 city models (LOD0–4), catalogues what is published, and feeds a Cesium frontend with 3D Tiles plus the underlying data. First use: NARAGA / OIKN (checkpoint 13 Oct 2026); afterwards a template for other regions and themes. Stakeholders: NARAGA product owner, frontend (Cesium) developers, data producers, non-IT data editors, DevOps.

Requirements

Functional - F1 import CityGML with traceability · F2 catalog datasets/layers/tilesets · F3 API for frontend: layers by `lod/bbox/status/theme`, tileset, feature attributes · F4 non-IT CRUD with inventory and history · F5 mandatory user on every change · F6 one-command setup, Railway deploy · F7 reusable for a new project by changing env + seeds. **Non-functional** - Integrity: rules in DB. Auditability: every change logged with user. Portability: Docker; no vendor lock (Cesium ion or self-hosted tiles). Performance (to be measured, proposal): catalog endpoints p95 < 300 ms, feature lookup p95 < 500 ms for ≤ 5 M objects. Recoverability: nightly dump + tested restore (RPO 24 h, RTO < 2 h). Security: internal accounts, HTTPS, read-only API behind CORS. Maintainability: SQL-first migrations, pinned versions.

Component view (C4 container)

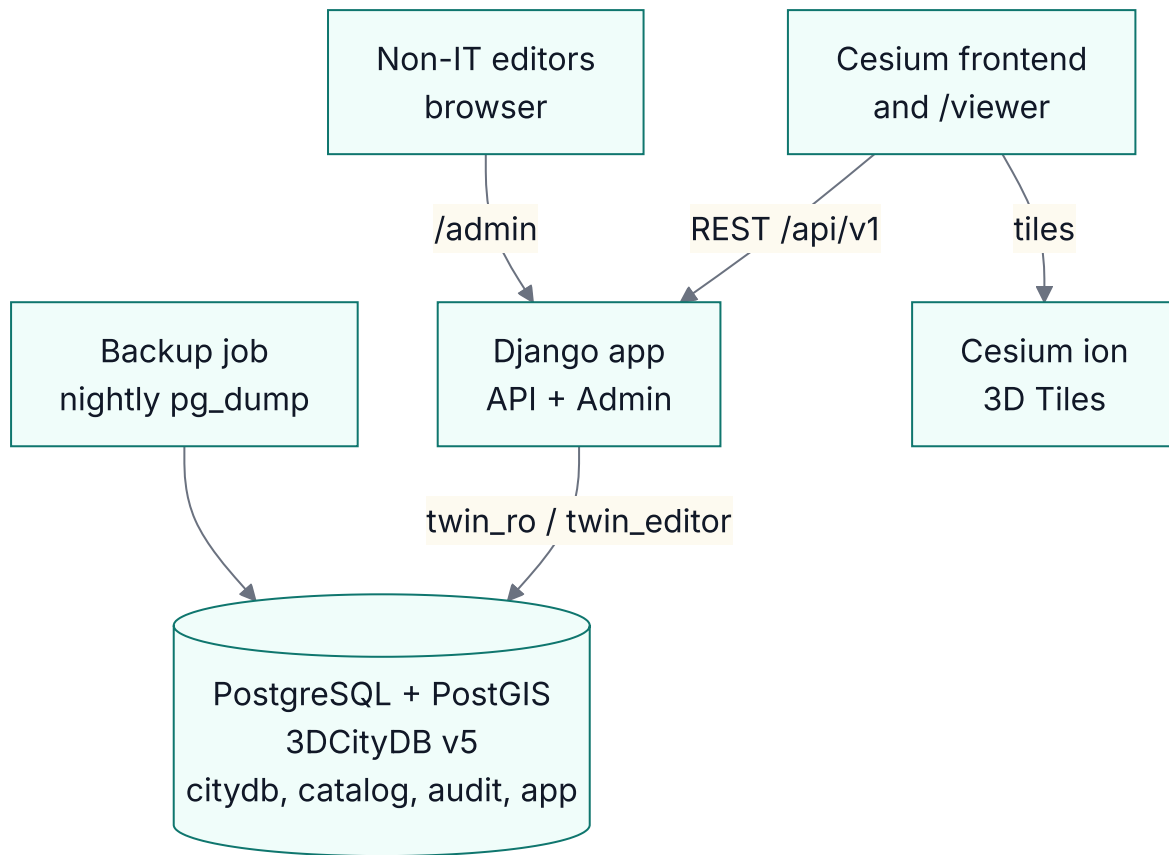


Figure 1. Runtime components. Everything inside the application boundary runs as Docker containers locally and as Railway services in production.

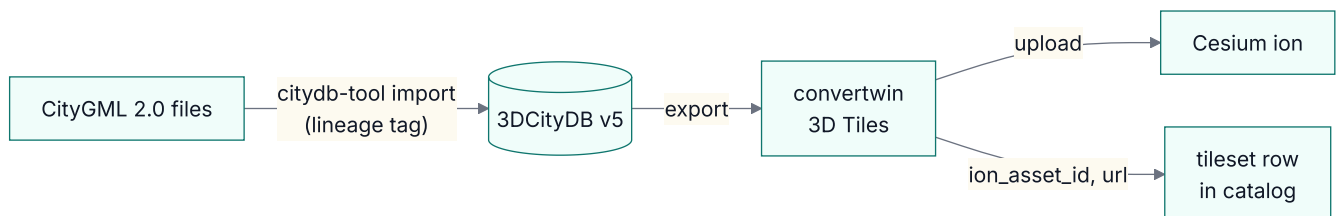


Figure 2. Build-time pipeline. citydb-tool and convertwin are command-line jobs, not running services.

Component	Responsibility
PostgreSQL/PostGIS + 3DCityDB v5	City model, catalog, audit, rules (triggers/constraints)
Django app	Read-only API (/api/v1), admin UI (CRUD, inventory, history), static /viewer/ ; sets app.user per transaction
citydb-tool	CLI job: import/export/delete CityGML (not a service)
convertwin + Cesium ion	Build/host 3D Tiles; result recorded in tileset
Backup job	Nightly pg_dump -Fc to an S3-compatible bucket
Postman/newman	API contract and post-deploy smoke tests

Deployment view

Local: `docker-compose.yml` (db, api, optional backup). Railway: same Dockerfiles, services `db` (3DCityDB image + persistent volume, private network) and `api` ; migrations + seeds run as pre-deploy command; healthcheck `/api/v1/health` . Details and unverified points in [deployment.md](#).

Data flow

1. **Import:** `make import USER=name FILE=x.gml DATASET=oikn THEME=building LOD=1` → job row (running) → `citydb import citygml --lineage oikn.building.lod1 --reason-for-update import_job:<id> --updating-person name` → `val3dity/counts` → job succeeded, `validation_passed` → `refresh_derived()` → link `import_job_layer` → layer validated.
2. **Convert/publish:** export or read from DB → `convertwin` → 3D Tiles → Cesium ion → insert `tileset ( ready , ion_asset_id , source_snapshot_at )` → activate → layer published.
3. **Serve:** frontend `GET /catalog?lod=1` → layer + active tileset → `Cesium3DTileset.fromIonAssetId`. Click → `GET /features/{objectid}` → attributes from `citydb`.
4. **Edit:** editor changes catalog fields in admin → trigger stamps user, logs history.
5. **Stale:** a later change to features (`last_modification_date > source_snapshot_at`) marks the layer stale → rebuild tiles.

Architecture decisions (ADR)

#	Decision	Why / consequence
001	3DCityDB v5.1.4 , CityGML 2.0 imported as-is (option A), exported as 2.0	Fixed versions keep semantics; LOD4 stays as deprecated properties. Round-trip test = counts per class/LOD/rooms + area/volume; passed on a synthetic sample, real LOD4 sample pending. Fallback v4 if it fails
002	Catalog in separate schema <code>catalog</code> ; <code>citydb</code> untouched but one index	3DCityDB upgrades don't break catalog; re-run migration after upgrade
003	Layer = (dataset, theme, LOD), a publishing slice; version on tileset	A feature can carry several LODs; feature history is in 3DCityDB
004	Feature↔dataset binding by <code>feature.lineage</code> tag; no link table	Native, indexed, no millions of link rows
005	One database per project	SRID fixed per DB; own backup/lifecycle; repo+seed is the template
006	Reference tables (not enums) for theme/status	Admins extend without migrations
007	Business rules and mandatory user in the database	Every entry point obeys them
008	Archive via status; admin-only delete; JSON audit	No partial-index sprawl
009	Django admin + django-ninja in one project	Real-user audit, non-IT CRUD, one stack. Rejected: NocoDB (single DB role hides user; license), Directus (schema handling, license), Baserow (no existing schema)
010	dbmate SQL-first migrations	Simple binary, plain SQL, template-friendly
011	Tiles on Cesium ion; <code>provider</code> allows self-hosted	Matches current flow; no lock-in
012	Tile objects carry <code>objectid</code> (gml:id) so click → <code>/features/{objectid}</code>	Tile ↔ CityDB integration. Unverified: that convertwin preserves ids (check 9 Oct); fallback own writer
013	Geometry is never edited in the UI; edits via 2D source → regenerate → delete by lineage → re-import	3D editing is unsafe for non-IT users (post-checkpoint)

#	Decision	Why / consequence
014	Plain internal accounts (Django) now; NARAGA SSO later through the same <code>app.user</code> mechanism	Internal tool, simple
015	Railway as target host, Docker-portable	Dockerfiles per service, settings in the dashboard (railway.toml is deprecated); no Railway-only features in code

Security and access

Admin/viewer: Django username/password (admin-created, one per person), groups viewer/editor/admin, HTTPS. DB: least-privilege roles; the API connects as `twin_ro`, admin as `twin_editor`. API read-only, unauthenticated at checkpoint, CORS allow-list via env. Cesium ion token only in frontend/viewer (browser storage), never in API or DB. Secrets only in env vars, `.env` git-ignored. DB not exposed publicly in production.

Risks and deferred

Risk	Mitigation
Height datum mismatch (buildings float/sink)	<code>vertical_datum</code> , <code>tileset.height_offset_m</code> ; test 9 Oct
convertwin drops feature ids	Verify early; fallback own converter step
Duplicate <code>gml:id</code> across datasets with import modes that match by id	Unique ids/prefix; default <code>import_all</code> ; delete by lineage
SRID chosen wrongly at DB creation	Confirm source CRS before first import; recreate is cheap early
Railway volume limits/cost/backups	Verify before 7 Oct; own bucket backups
Cesium ion quota	Check before 9 Oct
LOD1 data late (6 Oct)	Critical path; viewer + API proceed in parallel
3D quality (<code>ST_IsValid</code> is 2D only)	Use <code>val3dity</code> as gate

Deferred: region hierarchy, tags, ADE hooks, approval workflow, point-in-time recovery, staging schema, NARAGA token auth, feature-attribute editor views (`editor.*`), 2D `source` schema, CI pipeline.